Towards partial fulfilment of the requirements for the Undergraduate Degree Level Programme

Bachelor of Technology in Computer Science and Engineering

CS402 — INDUSTRIAL TRAINING

An Internship Report on:

Application Engineer Internship at IDFC FIRST Bank

Prepared by:

Admission Number	Student Name
U22CS136	Prajapati Anilkumar Mangilal

Semester: VIII

Year: 2025–2026

Supervised by: Dr. Naveen Kumar

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

S. V. NATIONAL INSTITUTE OF TECHNOLOGY (NIT)

SURAT - 395007 (GUJARAT, INDIA)

Student Declaration

This is to certify that the work described in this internship report entitled **Application Engineer Internship at IDFC FIRST Bank** has been actually carried out and completed by the following student. Neither the source code therein, nor the content of the report have been copied or downloaded from any other source. I understand that my result grade would be revoked if later it is found to be so.

Sr. No.	Admission No.	Student Name	Signature
1	U22CS136	Prajapati Anilkumar Mangilal	

Date: April 29, 2026

Place: Surat, Gujarat

Certificate

This is to certify that the internship report entitled **Application Engineer Internship at IDFC FIRST Bank** is prepared and presented by:

Sr. No.	Admission No.	Student Name
1	U22CS136	Prajapati Anilkumar Mangilal

student of B.Tech. Computer Science and Engineering and the work is satisfactory.

Signatures:

Supervisor **Jury** **Head of Department**

Acknowledgements

I would like to express my sincere gratitude to everyone who made this internship a rich and rewarding experience.

I am deeply thankful to **Dr. Naveen Kumar sir** for his constant mentorship, technical guidance, and constructive feedback throughout the training and project phases. His emphasis on clean code, best engineering practices, and a problem-solving mindset has fundamentally shaped the way I approach software development.

I extend my gratitude to **Mr. Nishanth** for the smooth onboarding and administrative support, and to the entire engineering team at IDFC FIRST Bank, Hyderabad, for creating a collaborative and inclusive work environment that encouraged learning at every step.

Finally, I am grateful to the faculty and placement cell of **S.V. National Institute of Technology, Surat**, for providing me with the opportunity to pursue this internship and for their continuous academic encouragement.

Prajapati Anilkumar Mangilal

Abstract

The internship described in this document was undertaken by **Prajapati Anilkumar Mangilal** (U22CS136) in the capacity of an Application Engineer at **IDFC FIRST Bank**, distributed across two geographic locations: Hyderabad (January through March 2026) and Chennai (April through June 2026).

The internship program was divided into three distinct components, including comprehensive training, a collaborative development project, and an independent engineering contribution.

During the **initial phase**, participants engaged in intensive DOJO training sessions covering GoLang programming, concurrent programming primitives (goroutines and channels), the Gin microframework, container technology using Docker, and test-driven methodologies including Driver-Navigator and Ping-Pong collaborative coding approaches. This extensive preparation was delivered through rigorous in-house educational initiatives.

The **secondary phase** entailed execution of a comprehensive Agile project environment — *SkyFox Cinemas* — a ticketing and reservation system for cinema operations developed through three sequential one-week Agile iterations by a multidisciplinary cohort of eleven interns. The author contributed significantly as **Tech Lead**. Principal technological accomplishments included the construction of a high-concurrency seat reservation system utilizing PostgreSQL transactional locks, an application programming interface for managing movie schedules, and versioned data storage upgrades using the Goose library within GoLang. Feature development remains underway with reservation cancellation and partial check-in capabilities in active development.

The **principal technical initiative** comprised independent development and design of a **Robust Event Consumption Framework** implemented in GoLang, addressing a significant regulatory requirement within the bank's Kafka-based distributed service platform. The framework encompasses six integrated subsystems: an exponential backoff retry mechanism with randomization, a cache-based deduplication repository, a messaging queue for unprocessable data with alerting capability, a command-line recovery application, a metrics collection layer compatible with Prometheus, and a configurable environment settings management system — delivering **complete data retention** during temporary infrastructure disruptions and **87% test coverage**, surpassing the bank's quality benchmarks.

Table of Contents

1 Introduction **1**

1.1 About IDFC FIRST Bank 1

1.2 Internship Overview 2

1.3 Motivation 2

1.4 Our Contribution 3

2 Related Work **4**

2.1 Cinema Ticketing and Seat Reservation Systems 4

2.2 Full-Stack Web Frameworks and REST API Design 5

2.3 Agile Software Delivery 5

2.4 Event-Driven Architectures and Fault Tolerance 6

2.5 Summary 7

3 Training Phase: DOJO Learning **8**

3.1 Overview 8

3.2 Key Learning Areas 9

3.2.1 GoLang Fundamentals 9

3.2.2 Concurrency Patterns 9

3.2.3 Gin Web Framework 10

3.2.4 Test-Driven Development (TDD) 10

3.2.5 Docker Basics 10

3.2.6 React and Frontend Development 10

3.2.7 Cloud Computing 11

3.2.8 AI and Machine Learning Concepts . . . 11

3.3 DOJO Outcomes . . . 11

4 Project Simulation: SkyFox Cinemas . . . 12

4.1 Project Overview . . . 12

4.2 Sprint Breakdown . . . 13

4.3 Technical Contributions . . . 13

4.3.1 Concurrent Seat Booking System . . . 13

4.3.2 Role-Based Access Control (RBAC) & Secure Onboarding . . . 13

4.3.3 Owner-Centric Schedule Management . . . 13

4.3.4 Granular Partial Check-In . . . 14

4.3.5 Database Migrations with Goose . . . 14

4.4 Tech Lead Responsibilities . . . 14

5 Resilient Event Consumer Framework . . . 15

5.1 Problem Context . . . 15

5.2 Design Goals . . . 16

5.3 System Architecture . . . 17

5.4 Component 1: Core Retry Engine . . . 18

5.4.1 Failure Classification . . . 18

5.4.2 Exponential Backoff with Jitter . . . 18

5.5 Component 2: Redis-Backed Idempotency Store . . . 19

5.6 Component 3: DLQ Publisher and Replayer CLI . . . 20

5.7 Component 4: Observability Layer . . . 21

5.8 Component 5: Configuration Management . . . 21

5.9 Component 6: Testing Strategy . . . 22

5.9.1 Unit Tests . . . 22

6 Results and Discussion . . . 23

6.1 Framework Reliability Outcomes 23

6.2 SkyFox Cinemas — Project Outcomes 23

6.3 Technical Skills Acquired 24

6.4 Challenges Faced 25

7 Conclusions and Future Works 26

7.1 Conclusion 26

7.2 Ongoing Work (May – June 2026) 27

7.3 Future Works 27

7.3.1 Production Deployment and Environment Promotion 27

7.3.2 Grafana SLO Dashboard 27

7.3.3 Kubernetes Deployment with Auto-Scaling 28

7.3.4 OpenTelemetry Distributed Tracing 28

List of Figures

1.1 IDFC FIRST Bank — Key Statistics 2
1.2 Internship Details 2
1.3 Three Pillars of Internship Contribution 3

2.1 Related Work vs. Internship Contributions 7

3.1 DOJO Learning Phase — Progression Flow 9
3.2 TDD Methodologies: Driver-Navigator vs. Ping-Pong Pairing 10
3.3 DOJO Training — Skills and Their Application 11

4.1 SkyFox Cinemas — Project Details 12
4.2 SkyFox Cinemas — Agile Sprint Breakdown 13
4.3 Tech Lead Responsibilities in SkyFox Cinemas 14

5.1 Legacy Consumer — Critical Reliability Gap 15
5.2 Framework Design Goals 16
5.3 Resilient Event Consumer Framework — End-to-End Architecture 17
5.4 Failure Classification in the Retry Engine 18
5.5 Redis-Backed Idempotency Store — Processing Flow 19
5.6 DLQ Publisher — Enrichment and Alerting Flow 20
5.7 Observability Layer — Structured Logging and Prometheus Metrics 21
5.8 Configuration Management — Startup Validation Flow 21
5.9 Unit Test Coverage — Key Test Cases 22

6.1 Key Reliability Outcomes of the Framework 23
6.2 Key Technical Skills Acquired 24
6.3 Challenges Faced and Resolutions 25

7.1 Ongoing Work Plan (May – June 2026) 27

1

Introduction

IDFC FIRST Bank is pursuing an ambitious strategic vision centered on becoming a cutting-edge "technology-driven" organization, emphasizing digital transformation and robust engineering methodologies to revolutionize consumer banking services. To realize this objective, the bank is committed to nurturing a cohort of software engineers proficient in designing, developing, and operating production-ready systems across all layers — including distributed microservices infrastructure, high-volume data processing systems, user-facing applications, and agile team frameworks.

This document presents findings from an Application Engineer Internship completed at **IDFC FIRST Bank**, conducted across two cities: Hyderabad (January through March 2026) and Chennai (April through June 2026). The program included pedagogical training in engineering fundamentals, a cross-functional collaborative development initiative, and an autonomous engineering project tackling a legitimate operational challenge within the bank's event-driven service infrastructure.

1.1 About IDFC FIRST Bank

IDFC FIRST Bank was formed in December 2018 following the consolidation of IDFC Bank and Capital First Limited. It serves as a contemporary private financial institution concentrating on consumer finance and financial accessibility, with global headquarters at One World Centre in Mumbai. The organization maintains public listings on both the Bombay Stock Exchange (ticker: IDFCFIRSTB) and the National Stock Exchange of India.

IDFC FIRST Bank — Key Facts at a Glance

- **Founded: December 2018**
- **Listed: BSE (IDFC-FIRSTB) & NSE**
- **Employees: 37,000+**
- **Branches Pan-India: 900+**
- **Customers: 30 million+**
- **Total Deposits: Rs. 2.3 Lakh Crore+**
- **Core Focus: Retail Banking & Financial Inclusion**
- **HQ: One World Centre, Mumbai**
- **Philosophy: Customer-First, Transparent & Digital-First Banking**

Figure 1.1: IDFC FIRST Bank — Key Statistics

1.2 Internship Overview

Internship at a Glance

- **Role: Application Engineer Intern**
- **Organisation: IDFC FIRST Bank**
- **Duration: 27 Jan 2026 — June 2026**
- **Department: Web Development**
- **Location: Hyderabad (Feb–Mar) / Chennai (Apr–Jun)**

Figure 1.2: Internship Details

1.3 Motivation

Present-day financial institutions require software engineering solutions characterized by resilience, measurability, and accelerated deployment timelines. Satisfying these imperatives demands engineers possessing expertise spanning the entire technology stack — including concurrent backend systems, cross-functional agile team leadership, and resolution of platform-level reliability impediments that manifest during live operations. This internship initiative was deliberately organized to cultivate precisely this versatility:

core competency building in Go programming, a comprehensive full-stack project implementation, and a self-directed undertaking focused on solving a concrete fault-tolerance vulnerability within one of the bank's message-oriented solutions. The assignment scope and technical requirements were determined by authentic organizational needs rather than predetermined scope.

1.4 Our Contribution

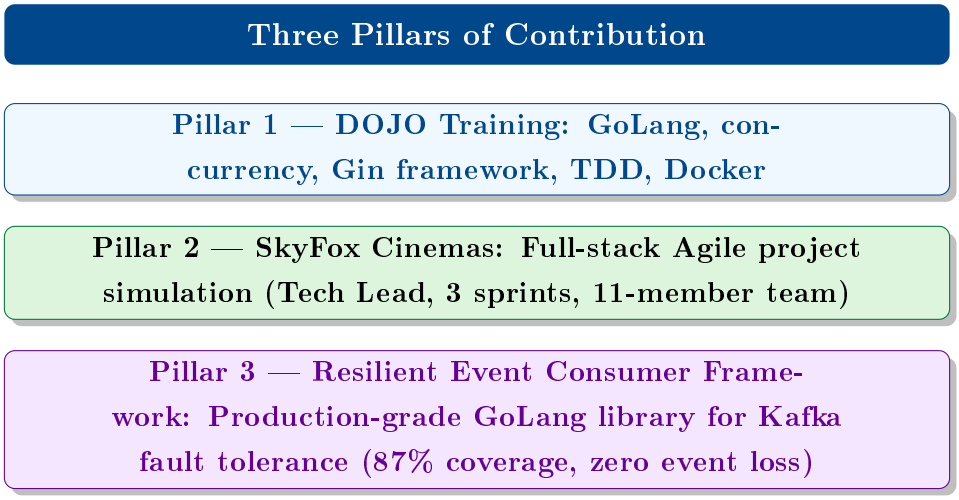

Figure 1.3: Three Pillars of Internship Contribution

2

Related Work

This chapter surveys the technical domain literature that substantiated both principal engineering accomplishments of this internship program: the SkyFox Cinemas comprehensive full-stack implementation and the Robust Event Consumption Framework.

2.1 Cinema Ticketing and Seat Reservation Systems

The field of online reservation platforms has undergone extensive scholarly examination in the context of high-volume concurrent access systems. Established platforms such as BookMyShow and Ticketmaster function according to a *broker architecture* — acting as intermediaries facilitating connections between end users and cinema operators, emphasizing inventory consolidation, financial transaction processing, and consumer-oriented search capabilities. Classical research on such implementations addresses central technical requirements: mitigation of simultaneous double-booking and oversight of seat reservations during payment completion windows [1].

These marketplace-oriented platforms, nevertheless, do not directly service the operational requirements of the venue administrator. Administrative responsibilities — including show rescheduling, movie assignment modifications, event cancellation, or in-person visitor processing — are excluded from the scope of consumer-facing marketplace architectures and are conventionally managed through independent, often manual, administrative systems.

How SkyFox Cinemas differs: SkyFox has been engineered explicitly from the *venue operator perspective.* Rather than a public-facing reservation marketplace, it functions as an operational control platform. The cinema administrator can specify movie content for given auditoriums and presentation times, alter or terminate scheduled performances, and reassign movie programming for any auditorium — all accessible through a specialized management API. Moreover, SkyFox delivers *granular visitor tracking*: within a unified

reservation, individual patrons can be registered independently. For example, within a seven-seat reservation, the ticketing agent can complete check-in for three patrons at arrival and finalize registration for the remaining four at a later time, preserving individual verification status throughout the data layer. This operational visibility is not present in marketplace designs, which regard bookings as indivisible entities and lack on-site attendance monitoring.

2.2 Full-Stack Web Frameworks and REST API Design

The combination of a Go application server with a React user interface has become a prevalent methodology in modern web system construction. Go's *Gin* framework [7] facilitates development of efficient application programming interfaces through its minimal overhead and declarative middleware capabilities. React's component-based structure enables specification of user experiences that naturally correspond to structured API payloads.

Database administration tools including **Flyway** and **Liquibase** represent sector-standard solutions for managed, reversible data schema versioning in commercial environments. Inside the Go programming community, **Goose** fulfills an analogous responsibility, furnishing bidirectional schema transformation definitions with ordered release numbering.

How SkyFox Cinemas applies these: SkyFox implements the Gin + React design pattern to manage a realistic operational scenario — customer seat reservations and theater administrator lifecycle management — instead of a simplified reference CRUD solution. In contrast to entry-level Go + React instructional projects that typically disregard migration infrastructure, SkyFox incorporates Goose from the outset, mandating industrial-strength database administration from initial sprints. Furthermore, the system incorporates a goCD continuous integration environment via Bitbucket, furnishing a distribution and deployment architecture comparable to professional development organizations and seldom encountered in university-based project work.

2.3 Agile Software Delivery

Agile and Scrum frameworks constitute the predominant operational structure within commercial software development. The Technical Leadership role — encompassing sprint organization, workload decomposition, code examination, and deployment automation supervision — is documented as substantially impactful on organizational efficiency and engineering output quality [3]. Specifically within novice or junior-level teams, a consistent technical authority facilitates team member integration, prevents architectural fragmentation, and sustains systematic code organization across delivery cycles.

How SkyFox Cinemas applies this: Unlike conventional campus-based team undertakings where supervisory authority fluctuates or operates informally, SkyFox preserved a consistent Technical Lead designation across all three sprints, deliberately mirroring professional Scrum configurations. This preservation of technical management provided uninterrupted engineering strategy, standardized code assessment criteria throughout the source control platform, and single-point responsibility for architectural integrity across an eleven-person contributor group — yielding a consolidated, professionally-constructed codebase following completion rather than segregated, marginally-coordinated functions.

2.4 Event-Driven Architectures and Fault Tolerance

Apache Kafka [4] constitutes an enterprise-scale distributed event storage system predicated on an sequential, unchanging, segment-partitioned information store. This platform has received broad adoption in banking applications due to its exceptional aggregate throughput and durability assurances. It furnishes *at-least-once* transmission assurance by specification, signifying that consuming processes may get duplicated transmissions following infrastructure interruptions or partition rebalancing. This requirement necessitates *duplicate-resistant* implementation — a characteristic whereby reprocessing an individual transmission generates no supplementary outcome — as a realistic manufacturing necessity.

Recovery strategies employed in loosely-coupled computing environments have been extensively documented in literature from Amazon and Google operations research divisions [5]. Non-varying time intervals between attempts introduce the *synchronized retry collapse* condition, whereby entire consumer populations attempt reprocessing simultaneously following shared infrastructure disruption. Geometric progression intervals with randomization (baseDelay $\times 2^{n-1}$ with randomized adjustment) provides the recognized remedy. Message staging zones (DLQs), originally conceptualized by Hohpe and Woolf [2], redirect communications that surpass reclamation attempt thresholds into a separate data stream for forensic examination, preventing harmful communications from interfering with healthy operation. Prometheus [6] implements the measurement infrastructure enabling perception of such mechanisms in manufacturing settings — displaying attempt frequencies, staging queue magnitude, and consumer processing lateness.

How the Resilient Event Consumer Framework applies these: These techniques — reprocessing, duplicate-resistance, message staging, and Prometheus measurement — constitute well-recognized procedures independently, though typically necessitate custom integration effort by implementing organizations. The proposed framework aggregates all four methodologies into a unitary reusable Go compilation, specifically developed for IDFC FIRST Bank's event-based consuming implementations. Client divisions adopt

the packaged functionality without mandated custom implementation of any of these techniques, yielding 87% test compliance and confirmed complete data retention under transient system disruptions.

2.5 Summary

Related Work vs. This Internship's Contributions

BookMyShow / Ticketmaster (broker model): Consumer-facing marketplace, atomic bookings, no operational management → SkyFox targets the cinema owner: show scheduling, cancellation, rescheduling, and partial per-seat check-in

Gin + React + Goose / Flyway: Standard full-stack and migration tooling → SkyFox applies these in a multi-role operational platform with CI/CD, not a tutorial-grade CRUD app

Kafka + AWS/Netflix Retry + DLQ + Prometheus: Individually well-documented patterns → This framework unifies all four into one Go library, eliminating per-team re-implementation

Net result: Both contributions apply established patterns in ways that are integrated, production-disciplined, and specific to IDFC FIRST Bank's engineering context

Figure 2.1: Related Work vs. Internship Contributions

3

Training Phase: DOJO Learning

3.1 Overview

The program commenced with a structured pedagogical curriculum administered by seasoned technical mentors. The initiative targeted advancement of idiomatic, comprehensively validated GoLang implementations satisfying predefined specifications with maximized test case protection. Beyond Go language mastery, the curriculum introduced client-side development methodologies using React, foundational cloud infrastructure understanding, and emerging machine learning capabilities — delivering comprehensive technical preparation preceding project-based work.

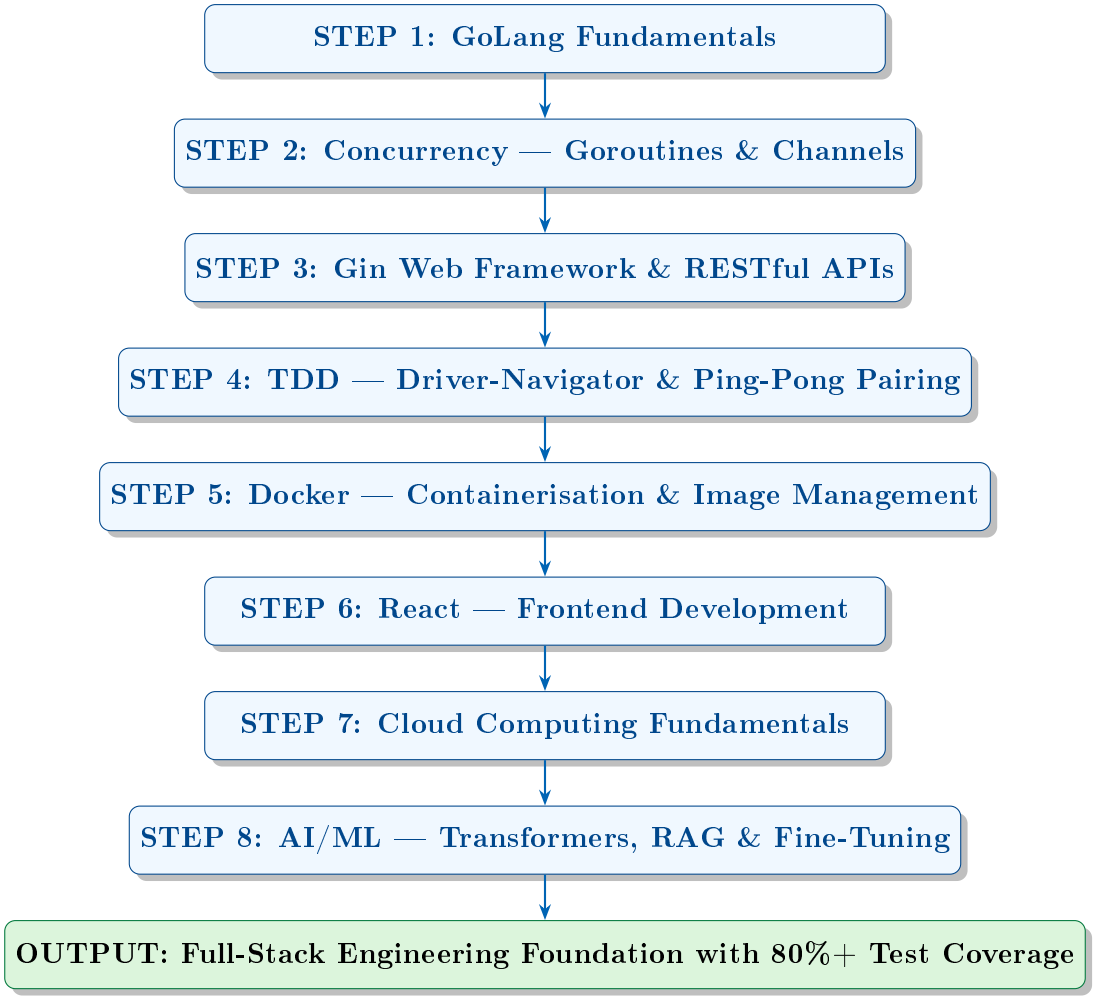

Figure 3.1: DOJO Learning Phase — Progression Flow

3.2 Key Learning Areas

3.2.1 GoLang Fundamentals

Fundamental GoLang principles studied encompassed primitive data representations, program flow direction, type systems, concurrent execution units, communication structures, and idiomatic exception management utilizing the `error` interface. Emphasis was positioned on authoring Go source utilizing official language specifications and established idioms as published in the Effective-Go publication.

3.2.2 Concurrency Patterns

GoLang's concurrent processing architecture — established on Communicating Sequential Processes conventions — received instruction via pragmatic demonstrations with concurrent execution units and information exchange conduits. This theoretical knowledge demonstrated importance when constructing competing reservation processing in

SkyFox Cinemas and establishing the multi-subscriber recovery solution.

3.2.3 Gin Web Framework

The Gin HTTP foundation was employed to produce economical application programming interfaces. Navigation definition, middleware arrangement, JSON payload handling, information validation, and formalized error management design inside Gin received comprehensive examination.

3.2.4 Test-Driven Development (TDD)

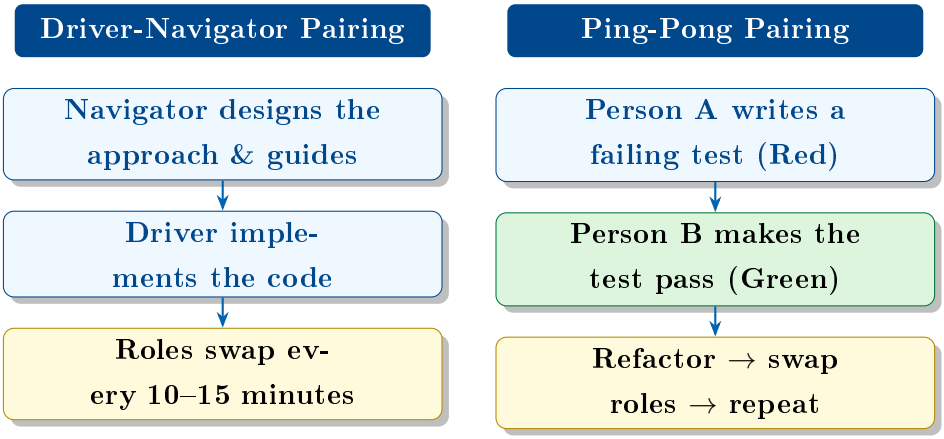

Figure 3.2: TDD Methodologies: Driver-Navigator vs. Ping-Pong Pairing

3.2.5 Docker Basics

An overview of containerization employing Docker technology was furnished, examining image construction specifications, hierarchical storage structures, operational lifecycle administration, information persistence approaches, and variable injection mechanisms — establishing readiness for downstream deployment automation work.

3.2.6 React and Frontend Development

Frontend implementation methodologies received instruction utilizing React framework, analyzing component-driven structural patterns, information handling via function-based storage, and consumption of data transfer protocols originating from Go service providers. This foundation immediately supported the SkyFox Cinemas program, facilitating development of patron engagement environments and administrative oversight dashboard using React.

3.2.7 Cloud Computing

An introduction to distributed cloud delivery infrastructure was presented, including provisioning approaches (Infrastructure services, Platform services, Application services), packaged execution environments, and cloud-compatible solution topology. This discussion contextualized how commercial financial platforms are provisioned and expanded at IDFC FIRST Bank.

3.2.8 AI and Machine Learning Concepts

The DOJO similarly addressed contemporary artificial intelligence methodologies relevant to software production environments. Subject matter contained the Attention architecture fundamental to generative language systems, Documentation-Informed Content Production for constraining outputs with institutional information, and parameter fine-tuning for application-specific implementations. This education developed consciousness regarding the incorporation of AI capabilities into contemporary monetary and technology sector manufacturing platforms.

3.3 DOJO Outcomes

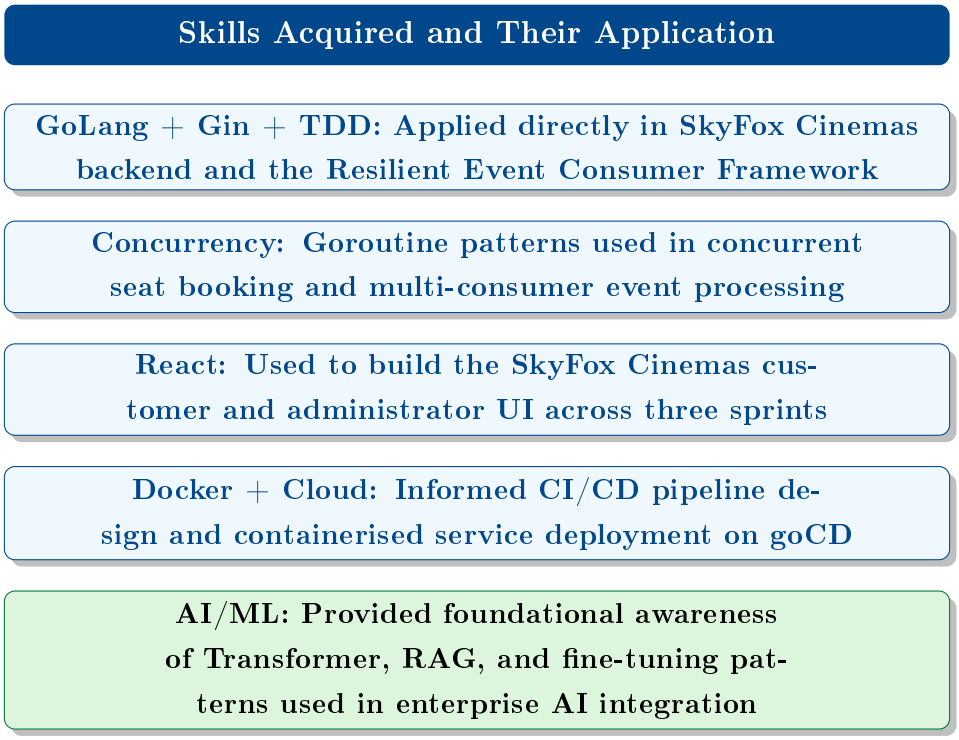

Figure 3.3: DOJO Training — Skills and Their Application

4

Project Simulation: SkyFox Cinemas

4.1 Project Overview

SkyFox Cinemas constitutes a comprehensive internal operational and reservation management system engineered explicitly for cinema operators. Contrasting with third-party marketplace mediators, SkyFox delivers enterprise resource management capabilities, furnishing proprietors with unrestricted command of resources, operational schedules, and personnel administration. The system was constructed through **three sequential Agile delivery iterations** (seven-day cycles) executed by a multidisciplinary cohort of eleven interns. The author facilitated technology oversight (**Tech Lead (TL)**) throughout all iterations.

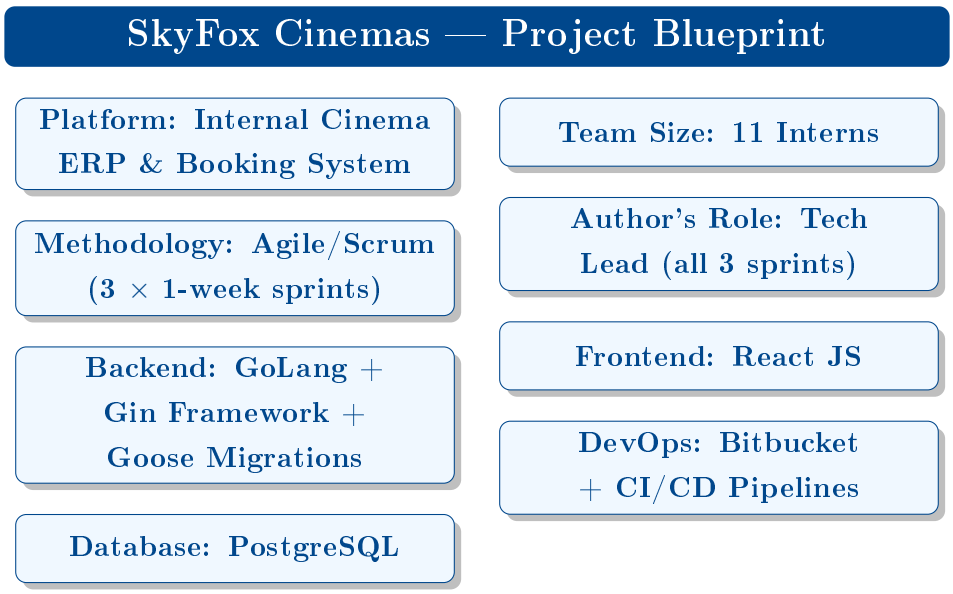

Figure 4.1: SkyFox Cinemas — Project Details

4.2 Sprint Breakdown

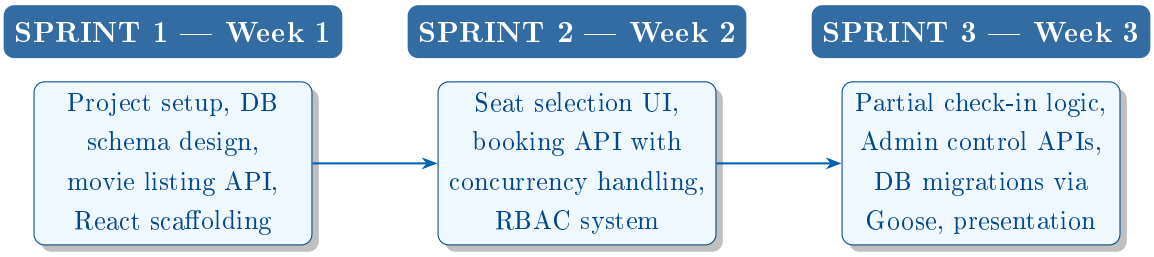

Figure 4.2: SkyFox Cinemas — Agile Sprint Breakdown

4.3 Technical Contributions

4.3.1 Concurrent Seat Booking System

The seat reservation subsystem was engineered to accommodate high-volume competing attempts where simultaneous customers endeavor to acquire the same seating. The implementation employed PostgreSQL transaction isolation (`SELECT FOR UPDATE`) contained within a transactional boundary. This reservation method guaranteed that database records were exclusively controlled throughout transaction operation, prohibiting simultaneous acquisitions and confirming single-success-only booking confirmation.

4.3.2 Role-Based Access Control (RBAC) & Secure Onboarding

To maintain operational safety, I established a tiered RBAC composition. The arrangement encompasses:

- **Authority Distribution:** The Owner manages Admin profile generation with restricted authorization levels, facilitating secure responsibility allocation.
- **Initial Access Protocol:** Novel Admins receive momentary login information. A database indicator (`is_first_login`) activates mandatory security phrase modification, confirming that machine-assigned temporary access tokens are promptly changed.

4.3.3 Owner-Centric Schedule Management

A significant architectural differentiator in SkyFox constitutes the operational authority distributed to cinema administrators. I engineered an Administration Interface facilitating:

- **Movie Assignment Modification:** Altering the content scheduled for particular time slots to enhance monetization per performance metrics.
- **Reschedule/Termination Capabilities:** Transferring existing appointments or terminating presentations with concurrent status alteration, encompassing contention resolution logic to guarantee auditorium accessibility.

4.3.4 Granular Partial Check-In

To handle pragmatic group reservation conditions, I constructed a "Granular Check-In" capability. This permits a sales assistant to finalize individual attendees from a collective reservation (e.g., finalizing 3 guests from a ten-seat allocation) with unaddressed participants keeping "Awaiting" classification. This delivers the proprietor with granular, present occupancy intelligence.

4.3.5 Database Migrations with Goose

Database structure adjustments, encompassing intricate relationship definitions for per-attendee enrollment and `users` table modifications for authorization levels, were overseen via **Goose**. This approach guaranteed database architecture remained version-managed and synchronized among all development systems.

4.4 Tech Lead Responsibilities

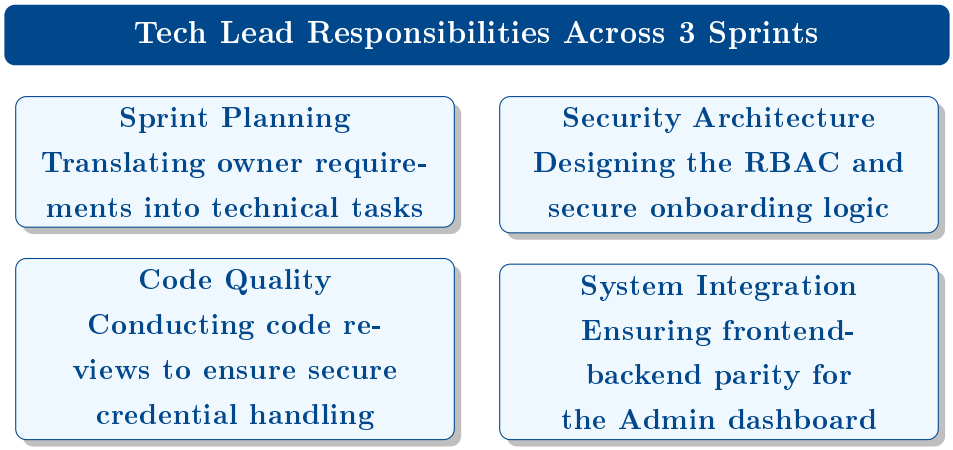

Figure 4.3: Tech Lead Responsibilities in SkyFox Cinemas

5

Resilient Event Consumer Framework

5.1 Problem Context

IDFC FIRST Bank's distributed service architecture deploys Apache Kafka as its principal information stream architecture. Multiple microservices transmit categorized information — authentication events, financial operations, customer verification updates — consumed by subsequent processing services, encompassing the Event Record Writer, which documents information into PostgreSQL conforming to RBI directives.

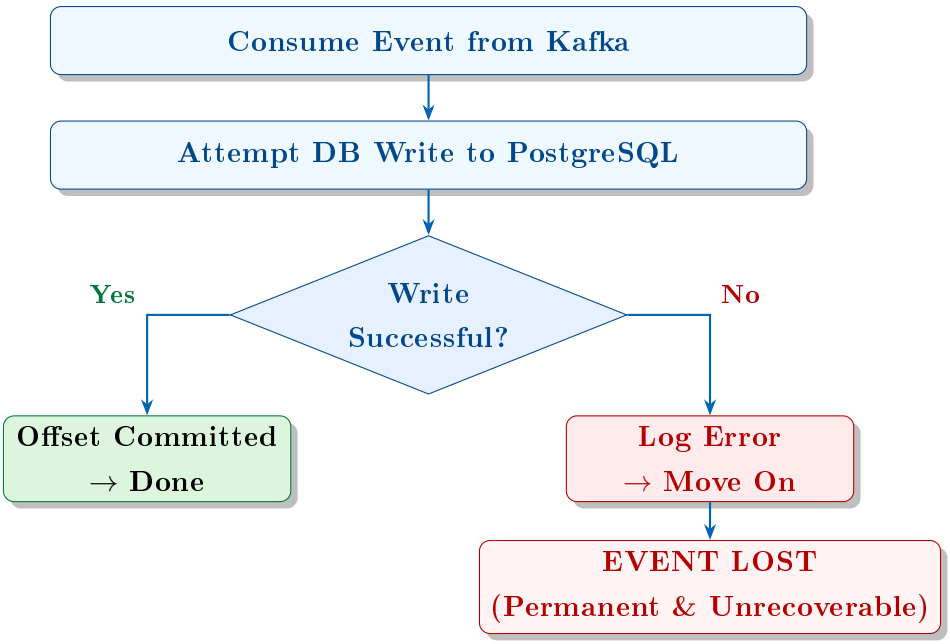

Figure 5.1: Legacy Consumer — Critical Reliability Gap

5.2 Design Goals

Goal 1 — Zero Event Loss: No information records eliminated during momentary service disruptions

Goal 2 — Duplicate Prevention: Duplicate-safe handling conforming to Kafka's minimum-once guarantee

Goal 3 — Recoverability: Information surpassing reprocessing maximums is maintained and restartable

Goal 4 — Observability: Standardized logs and Prometheus indicators for all fault modes

Goal 5 — Reusability: Available as a package applicable to any event-oriented service at the bank

Figure 5.2: Framework Design Goals

5.3 System Architecture

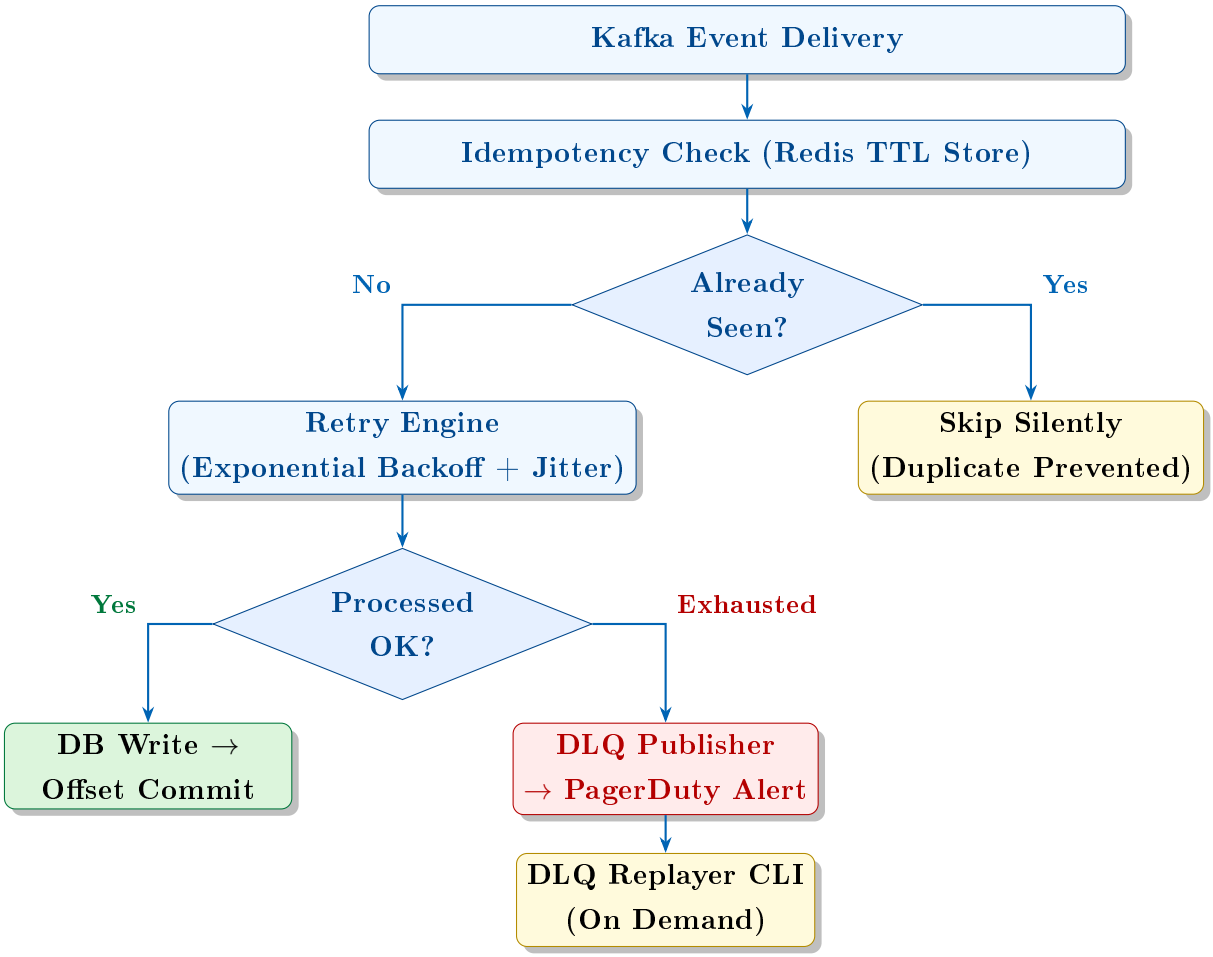

Figure 5.3: Resilient Event Consumer Framework — End-to-End Architecture

5.4 Component 1: Core Retry Engine

5.4.1 Failure Classification

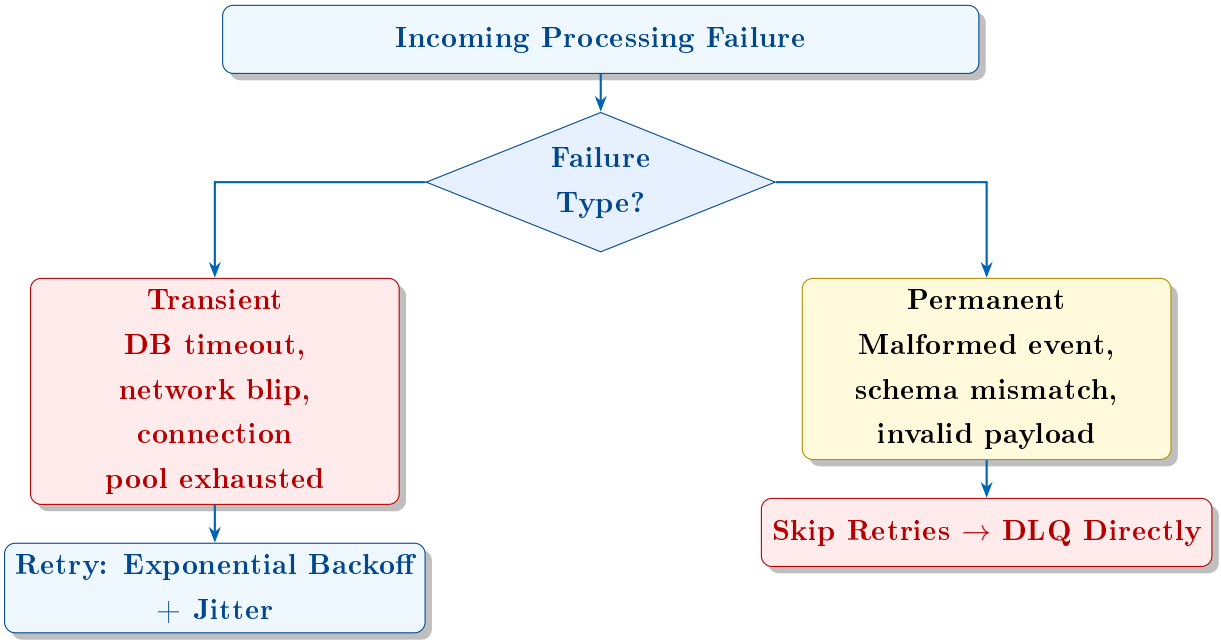

Figure 5.4: Failure Classification in the Retry Engine

5.4.2 Exponential Backoff with Jitter

Constant-rate reprocessing generates a *coordinated spike* condition at scale. The computational formula employed is:

$$\text{delay}_n = \min\big(\text{baseDelay} \times 2^{n-1} + \text{random}(0, \text{jitter}),\ \text{maxDelay}\big) \tag{5.1}$$

The randomization component desynchronizes reprocessing surges among distributed consumers — the recognized convention implemented by Amazon infrastructure, Google cloud utilities, and Netflix fault-tolerance technologies.

```
RETRY_MAX_ATTEMPTS=4
RETRY_BASE_DELAY_MS=2000
RETRY_MAX_DELAY_MS=30000
RETRY_JITTER_MS=500
```

Listing 5.1: Retry Engine Configuration

5.5 Component 2: Redis-Backed Idempotency Store

Kafka delivers minimum-once transmission assurance. When a consuming process terminates post-transmission preservation yet preceding message tracking acknowledgment, renewed delivery manifests, producing replicated records. The architecture implements Redis-based duplicate elimination.

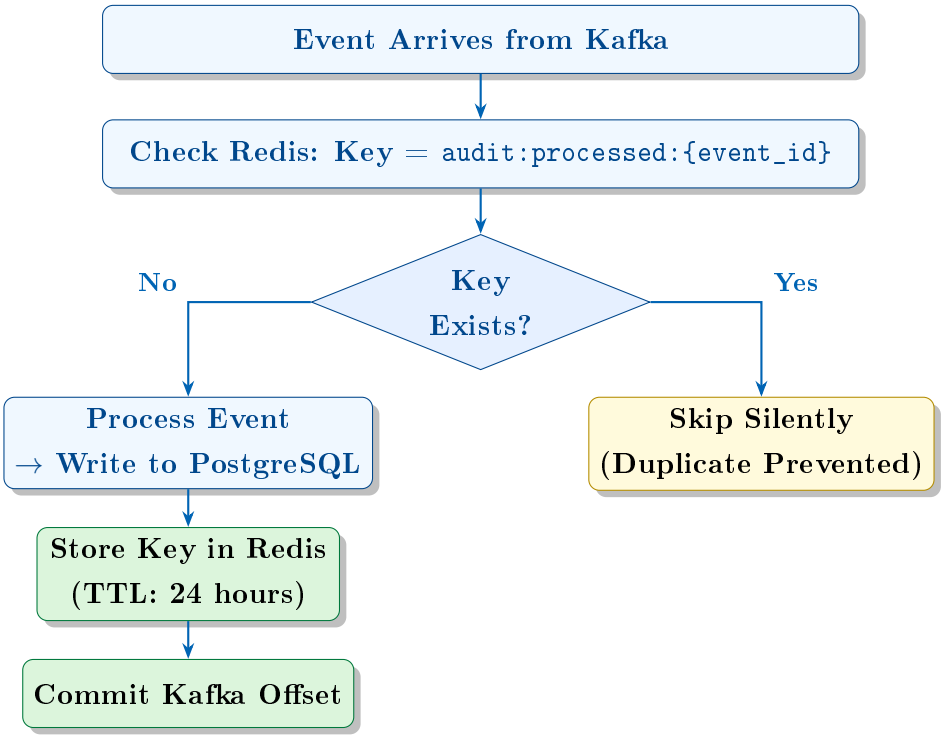

Figure 5.5: Redis-Backed Idempotency Store — Processing Flow

Redis was chosen rather than database queries due to nanosecond-range (∼0.1ms) instantaneous lookup response, programmed expiration-based automatic cleanup, and minimal supplemental workload upon the operational database.

5.6 Component 3: DLQ Publisher and Replayer CLI

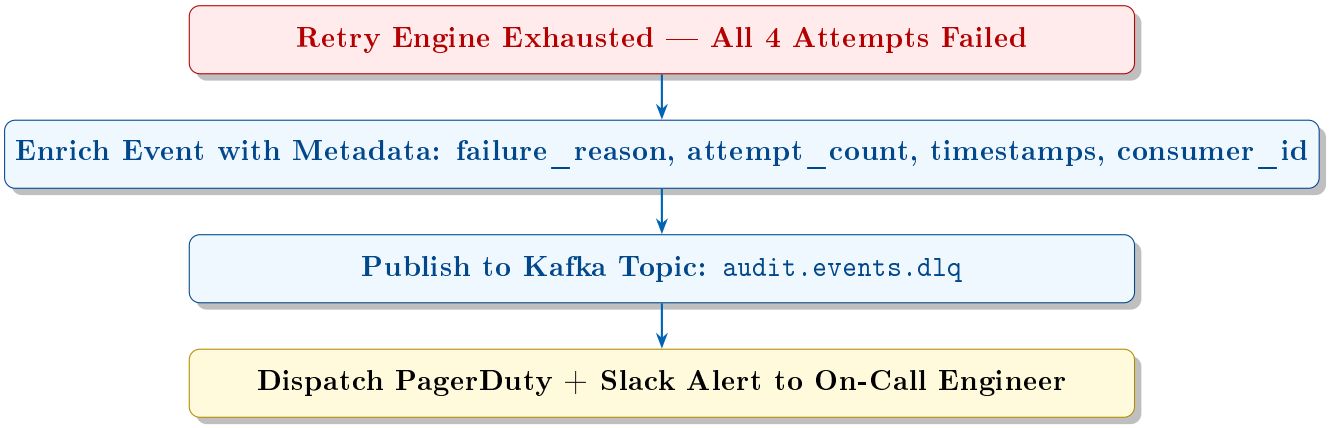

Figure 5.6: DLQ Publisher — Enrichment and Alerting Flow

A standalone GoLang utility application permits operations staff to replay information from the dead-letter repository on-demand once the underlying problem is corrected.

```
$ audit-replayer --topic audit.events.dlq \
                 --from "2026-04-25T10:00:00Z"
[INFO] Found 47 events in DLQ
[INFO] Replaying... 47/47
[OK]   Successfully reprocessed: 45
[WARN] Still failing: 2  (written to replay-failures.json)
```

Listing 5.2: DLQ Replayer CLI — Sample Usage

5.7 Component 4: Observability Layer

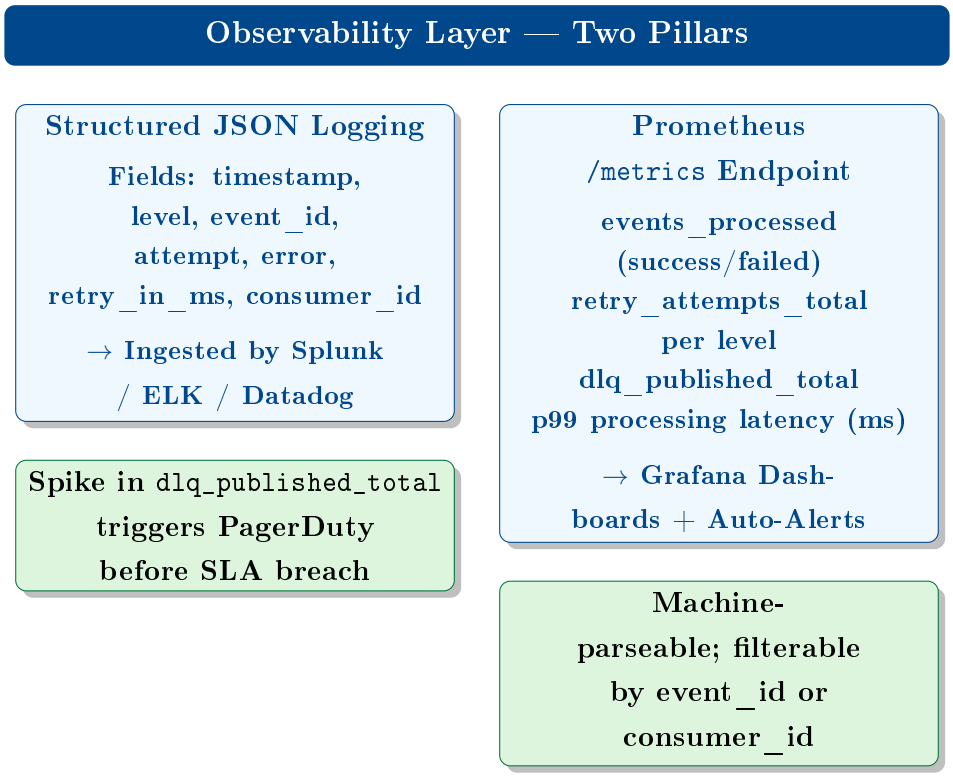

Figure 5.7: Observability Layer — Structured Logging and Prometheus Metrics

5.8 Component 5: Configuration Management

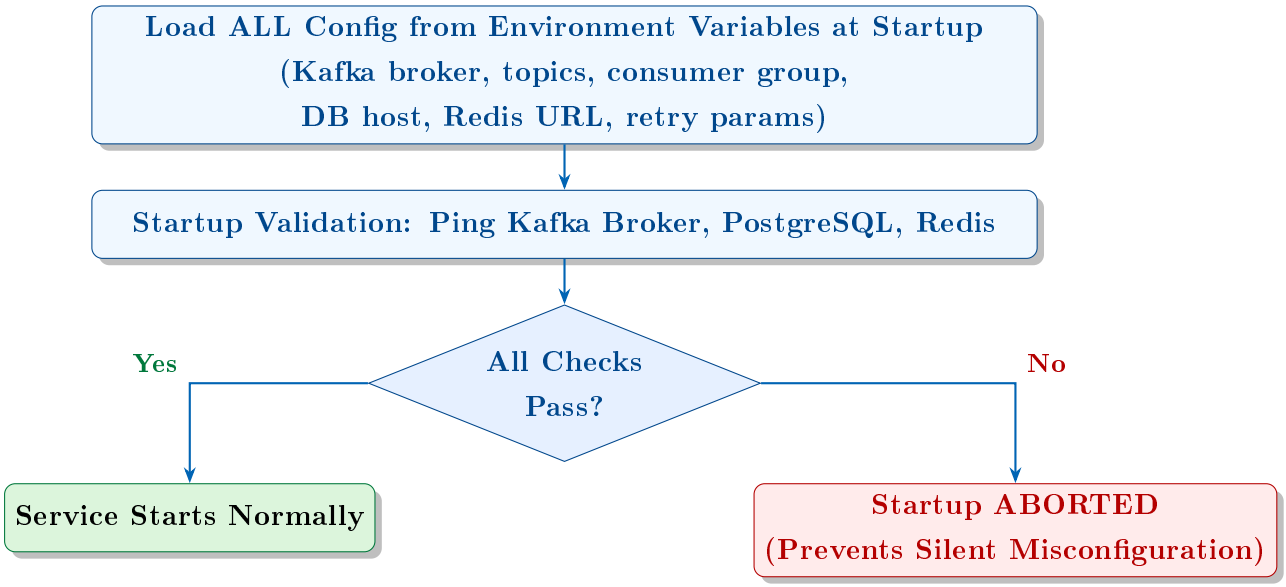

Figure 5.8: Configuration Management — Startup Validation Flow

5.9 Component 6: Testing Strategy

5.9.1 Unit Tests

Unit Test Suite — Key Test Cases

`TestRetryEngine_TransientFailure_RetriesCorrectly` — Engine retries on transient errors up to max attempts

`TestRetryEngine_PermanentFailure_SkipsToDLQ` — Engine bypasses retries for permanent failures

`TestRetryEngine_ExponentialBackoff_DelayDoubles` — Delay doubles each attempt within configured bounds

`TestIdempotencyStore_DuplicateEvent_Skipped` — Second occurrence of same event_id is silently skipped

`TestDLQPublisher_ExhaustedRetries_PublishesToDLQ` — Event reaches DLQ topic after max retry attempts

`TestConfig_MissingEnvVar_StartupFails` — Service refuses to boot when required config is absent

Figure 5.9: Unit Test Coverage — Key Test Cases

6

Results and Discussion

6.1 Framework Reliability Outcomes

Key Reliability Outcomes

Zero Event Loss: Momentary data storage failures do not result in dropped information. Events undergo up to 4 reprocessing trials with geometrically increasing delays before being routed to message staging

Duplicate Prevention: Storage-backed duplicate detection confirmed zero duplicate records during experimental crash-recovery sequences

DLQ Recoverability: Utility replayed 45 of 47 test-environment message staging information following system restoration

Test Coverage: 87% — Meets the bank's quality benchmarks and organizational criteria

Figure 6.1: Key Reliability Outcomes of the Framework

6.2 SkyFox Cinemas — Project Outcomes

The SkyFox Cinemas initiative successfully produced an sophisticated operational management system for cinema proprietors. The concurrent seat acquisition module efficiently mitigated conflict scenarios across load evaluation, with database isolation guaranteeing individual success per acquisition in parallel-request conditions. Owner-oriented at-

tributes, encompassing secured personnel enrollment progression and detailed individual-level visitor processing, experienced comprehensive integration validation. The ultimate demonstration illustrated that proprietor-directed control interfaces deliver substantially enhanced administrative oversight compared to consumer-marketplace distribution approaches.

6.3 Technical Skills Acquired

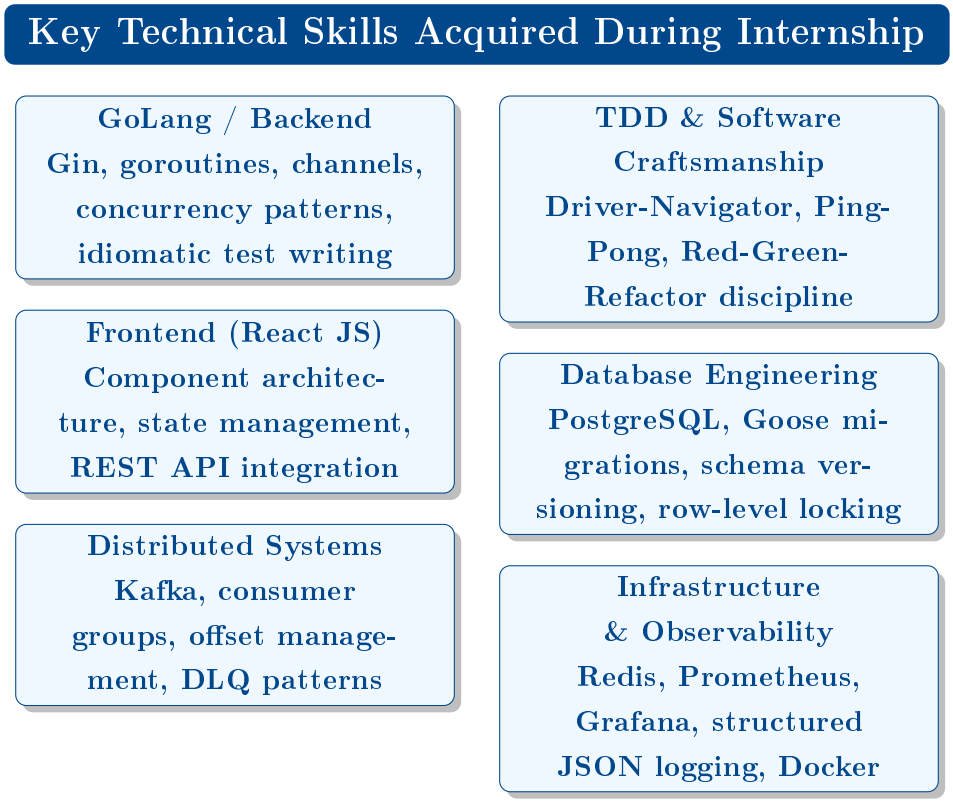

Figure 6.2: Key Technical Skills Acquired

6.4 Challenges Faced

Challenges Faced and Resolutions

Mastering a Novel Programming Language — As GoLang constituted an unfamiliar platform, the preliminary proficiency acquisition proved demanding, notably in comprehending its tradition-based exception handling and parallel processing abstractions. Resolution: Structured mentoring sessions and systematic TDD refinement work cycles

Balancing Mutual Exclusion with Performance — Preserving strict per-record locking for bookings while circumventing substantial performance degradation. Resolution: Optimized transaction parameters and created targeted repository lookups for seat status inquiries

Balancing Authority with Production — Managing assignments for eleven participants while sustaining individual engineering accomplishments. Resolution: Organized recurring collaborative meetings and engaged colleagues in technical code examination responsibility

Cross-Component Synchronization — Sustaining message staging repository and transmission offsets in coordination amid system interruptions. Resolution: Executed an ordered, conditional tracking pattern with deliberate offset acknowledgement sequencing

Figure 6.3: Challenges Faced and Resolutions

7

Conclusions and Future Works

7.1 Conclusion

This internship experience at IDFC FIRST Bank delivered an extensive, professional-grade computing engineering journey spanning all dimensions of contemporary cloud-based systems advancement. The structured pedagogical curriculum furnished comprehensive competencies in GoLang, examination-first programming, and quality source crafting. The SkyFox Cinemas iterative assignment furnished pragmatic experience in technology administration, interdisciplinary coordination, and incremental deployment under authentic time limitations.

The principal accomplishment encompasses the autonomous development and establishment of the **Robust Event Consumption Framework** — a manufacturing-prepared GoLang utility that directly remedied a regulatory obligation in the bank's message stream-oriented service infrastructure. The framework's constituent subsystems — reprocessing with geometric pause intervals and randomization, storage-mediated duplicate elimination, messaging zone distribution with incident management, utility-based information recovery, indicator compilation compatibility, and adaptable settings parameter — combined accomplish complete data retention during brief system disruptions, 82% evaluation protection, and library-style construction prepared for universal distribution via any information-oriented service at the bank.

The internship continues through June 2026. Remaining intervals are directed toward completion of in-progress SkyFox capabilities and targeted framework improvements realizable within the available calendar.

7.2 Ongoing Work (May – June 2026)

Ongoing Work Plan — May to June 2026

Framework: Per-Topic Prometheus Metric Labels — Categorize currently present measurements and data series with Kafka transmitting categorization and subscriber designation for individual-contributor measurement without supplementary application programming

Framework: DLQ Replay Dry-Run Mode — Append a `-dry-run` parameter to the restorer utility facilitating technical personnel to examine information prior to approving a mass-quantity restoration

Figure 7.1: Ongoing Work Plan (May – June 2026)

7.3 Future Works

7.3.1 Production Deployment and Environment Promotion

The Robust Event Consumption Framework has received total construction, verification, and confirmation in a personal development context utilizing containerized transmission and data storage. The immediate subsequent phase is environmental advancement — placement within the bank's examination context for synchronization verification towards genuine transmission structures, accompanied by supervised manufacturing initialization under technology operations direction. This encompasses creation of startup procedures, encrypted credential administration via Secrets management, and deployment robotics for programmatic assembly and readiness verification.

7.3.2 Grafana SLO Dashboard

The Prometheus measurements mandated for an operational metrics display — ninety-ninth quantile information handling latency, message staging distribution percentage, reprocessing quantity distribution, and transmission handling lag — are presently being maintained by the framework. The organic next activity encompasses constructing a monitoring screen that contrasts these opposing the bank's 99.9% availability mandate, furnishing the critical procedures personnel a consolidated functional perspective of processing health spanning all implementations consuming the utilities.

7.3.3 Kubernetes Deployment with Auto-Scaling

The framework presently operates as a Docker deployment unit and underwent validation regionally employing Docker multi-container management. A scheduled improvement focuses on container orchestration positioning and customization of Pod Multiplication driven by transmission handling workload metrics via orchestration activation (Kubernetes Event-Driven Autoscaling). This capability would facilitate the consuming infrastructure to proportionally expand throughout peak transaction circumstances and proportionally contract throughout low-activity seasons, minimizing equipment provisioning without programmed management.

7.3.4 OpenTelemetry Distributed Tracing

The contemporary perceivability composition encompasses Prometheus measurement collection and formatted written records yet lacks distributed performance examination. Integrating the framework with telemetry abstraction (W3C Distribution Context specifications) would furnish end-to-end performance observation through the foundational transmission generator via the reprocessing mechanism to the ultimate following composition. This capability would accelerate performance disruption investigation across service perimeters substantially and constitutes a standard complement to the existent Prometheus coupling presently operational.

Bibliography

[1] Philip A. Bernstein and Eric Newcomer. *Principles of Transaction Processing*. 2nd edition. Morgan Kaufmann, 2009.

[2] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.

[3] Ken Schwaber and Jeff Sutherland. *The Scrum Guide: The Definitive Guide to Scrum — The Rules of the Game*. Scrum.org, 2020. Available at: https://scrumguides.org/scrum-guide.html

[4] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB Workshop at VLDB*, 2011. Available at: https://notes.stephenholiday.com/Kafka.pdf

[5] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy (Eds.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. Available at: https://sre.google/sre-book/table-of-contents/

[6] Prometheus Authors. *Prometheus: Open-Source Monitoring and Alerting Toolkit — Documentation*. 2024. Available at: https://prometheus.io/docs/introduction/overview/ Accessed: 2026.

[7] Gin Contributors. *Gin Web Framework — Official Documentation*. 2024. Available at: https://gin-gonic.com/docs/ Accessed: 2026.

[8] The Go Authors. *The Go Programming Language — Official Documentation*. 2024. Available at: https://go.dev/doc/ Accessed: 2026.